Applied Stochastic Models: Hospital Patient Flow

Exhaustive Team Master Guide & Complete Technical Walkthrough

Course: Applied Stochastic Models (ASM) — Fall 2026	**GitHub:** github.com/Manas-Jain-24/applied-stochastic-models-hospital-flow
Empirical Grounding: 250,000 Real NYS SPARCS Records + MIMIC-IV Demo	**Verification Status:** 33 Automated Unit Tests (100% Passing in ~0.2s)

1. Executive Summary & Problem Formulation

Hospital management faces an intractable operational tradeoff: high occupancy is required to minimize capital waste, yet high occupancy causes catastrophic queueing—hallway emergency boarding, ambulance diversion, and postponed elective surgeries. Using classical and advanced stochastic processes, we built a fully operational, data-grounded computational pipeline that solves bed capacity sizing, queue prioritization, time-varying staffing, network patient routing, and optimal multi-department resource allocation.

---

2. Exhaustive Walkthrough of the 11 Research Questions

Q1: The Exponential Service Assumption & Heavy-Tail Delay Penalty

Theory & Test: The standard M/M/c queue assumes memoryless service with squared coefficient of variation cv^2 = Var(S)/E[S]^2 = 1.0. We fitted Exponential, Gamma, Lognormal, and Weibull distributions via Maximum Likelihood Estimation (MLE) on 250,000 real NYS SPARCS inpatient records.
Empirical Proof: Empirical E[S] = 5.386 days, Var(S) = 57.77 -> **cv^2 = 1.991 approx 2.0**. Kolmogorov-Smirnov test rejected the Exponential distribution with p < 10^-15 (AIC: 1,048,212 vs Lognormal AIC: 924,180).
Consequence (Allen-Cunneen M/G/c): Wq(M/G/c) approx Wq(M/M/c) * [(ca^2 + cs^2)/2] = Wq(M/M/c) * [(1 + 2.0)/2] = **1.50 * Wq(M/M/c)**. Patients experience a **50% longer waiting time** than textbook queuing predicts purely due to recovery variance!

Q2: Bed Capacity Sizing via Erlang-B Loss Modeling

Theory: For an offered load a = lambda * E[S] = (50 admissions/day) * (5.386 days) = 269.3 occupied bed-days. Running at 100% occupancy (c = 269 beds) causes an 8.2% patient rejection rate due to stochastic arrival bursts.
Algorithm: Erlang-B recursion B(c, a) = [a*B(c-1, a)] / [c + a*B(c-1, a)] with B(0, a) = 1.0.
Finding: Sizing for P_blocking ≤ 1% requires **c* = 293 beds**. The hospital requires a minimum safety buffer of **24 beds (+8.9%)** to absorb Poisson fluctuations without turning away patients.

Q3: Two-Class Non-Preemptive Priority Triage Queueing

Theory: Cobalt's priority queueing equations split traffic into Class 1 (Emergency: lambda_1 = 30/day, rho_1 = 0.528) and Class 2 (Elective: lambda_2 = 20/day, rho_2 = 0.352, total rho = 0.880). Residual service time W0 = sum(rho_i * E[Si^2]) / (2 * E[Si]).
Mathematical Formulas: Wq1 = W0 / (1 - rho_1); Wq2 = W0 / [(1 - rho_1)(1 - rho_1 - rho_2)].
Finding: Priority triage cuts Emergency wait time from 4.8 hours down to **1.5 hours (68.8% reduction)** by safely shifting waiting time onto scheduled elective patients (12.3 hours) with zero extra bed construction.

Q4: Non-Homogeneous Poisson Arrivals & Diurnal Thinning

Theory: Tested arrival rate stationarity using Index of Dispersion for Counts: IDC = Var(N(t)) / E[N(t)]. Stationary Poisson requires IDC = 1.0.
Finding: Empirical IDC = **1.34 > 1.0**, proving significant diurnal clustering. Arrivals follow an NHPP with rate function lambda(t) = 2.08 + 1.17 * sin(2*pi*(t - 8.2)/24). Demand peaks at **14:12 (2:12 PM) at 3.25 arrivals/hr** and drops to 0.91 arrivals/hr at 2:12 AM (a 3.6x peak-to-trough surge). Simulated exactly via Lewis-Shedler thinning algorithm.

Q5: Finite-Buffer M/M/c/K ED Holding & Ambulance Diversion

Theory: Real emergency departments have finite holding capacity K = c + B (c treatment bays, B boarding beds). When all K slots are filled, arriving ambulances are turned away (Ambulance Diversion). State probabilities computed using numerically stable Log-Sum-Exp normalizer.
Finding: Expanding ED holding buffer from B = 0 to B = 16 cuts ambulance diversion from **12.4% down to 2.8%**, at the expense of an average boarding holding wait of 3.6 hours, directly quantifying the trade-off between ambulance access and hallway boarding.

Q6: Absorbing Markov Chains for Downstream Patient Transitions

Theory: Hospital ward flow modeled as a discrete-time Markov chain with transient states (ED, ICU, Step-Down, MedSurg) and absorbing states (Discharge Home, SNF, Deceased). Canonical transition matrix P = [Q R; 0 I]. Fundamental matrix N = (I - Q)^-1 gives expected visits; B = N * R gives absorption probabilities.
Finding: An admitted ICU patient averages 1.48 ward transitions before exit, with a **10.1% in-hospital mortality rate**, 33.2% discharge to Skilled Nursing Facilities (SNF), and 56.7% discharge to home self-care.

Q7: Operational Stress-Testing & Epidemic Surge Collapse

Theory: Stress-testing the baseline hospital (c = 293 beds, mu = 0.18567/day) against sudden winter flu/COVID arrival surges.
Finding: A +10% surge elevates occupancy to 98.2% (waiting time jumps from 3.4 hrs to 18.7 hrs). A +20% demand surge elevates traffic intensity to **rho = 111.4% (> 1.0)**, causing mathematical system collapse (infinite queue backlog). Proves the necessity of automated trigger policies that pause elective admissions whenever occupancy reaches 95%.

Q8: Multi-Stage Open Jackson Queueing Networks

Theory: Jackson's Theorem proves product-form equilibrium for open networks: pi(n1,...,nK) = prod pi_k(n_k). Department arrival rates solve traffic balance equations: Lambda = gamma * (I - P^T)^-1.
Department Parameters: ED (c=12, mu=6/day), ICU (c=25, mu=0.25/day), StepDown (c=30, mu=0.33/day), MedSurg (c=120, mu=0.20/day).
Finding: MedSurg operates at **rho = 92.2%** (primary network bottleneck). Average total hospital dwell time across all departments is **5.19 days** via Little's Law.

Q9: Time-Varying PSA Queueing & Dynamic Shift Staffing

Theory: Pointwise Stationary Approximation (PSA) evaluates instantaneous traffic intensity rho(t) = lambda(t) / [c(t) * mu].
Optimization: Under constant staffing (c = 18 beds), afternoon arrival peaks push wait time to **41.5 minutes** at 15:00. By shifting 25 bed-shifts from quiet night hours (c = 14) to peak daytime hours (c = 22), peak wait time plunges to **1.1 minutes (a 97.3% reduction)** under **exactly zero additional bed-hours or labor cost**!

Q10: Multi-Department Optimal Bed Allocation via Fox's Greedy Algorithm

Theory: Allocate C_total = 160 beds across ICU (offered load a=22.0), ED (a=18.0), and Surgery (a=35.0) to minimize weighted blocking: min sum w_i * B(c_i, a_i) s.t. sum c_i = 160, where ICU blocking is penalized 10x (w_ICU = 10.0).
Algorithm: Fox's marginal allocation greedily assigns the next bed to the department yielding maximum marginal gradient Delta_i = w_i * [B(c_i, a_i) - B(c_i+1, a_i)].
Finding: Naive proportional allocation starves the ICU (c_ICU = 47 beds, 6.66% rejection rate). Optimal marginal allocation assigns **62 beds to ICU (+15 beds)**, slashing ICU rejection to **0.14%** and reducing total weighted blocking by **85.1%** (0.0655 vs 0.4037).

Q11: Modular MIMIC-IV & MIMIC-ED Pipeline Integration

Engineering: Built an automated adapter (src/mimic_adapter.py) parsing PhysioNet MIMIC-IV Clinical Demo v2.2 and MIMIC-IV-ED Demo v2.2. Extracts care-unit routing, ICU lengths of stay, and Emergency Severity Index (ESI 1-5 triage).
Status: Calibrated parameters saved in data/processed/mimic_empirical_parameters.json. When full credentialed PhysioNet access is granted, the exact same adapter processes full MIMIC data with zero code modifications.

3. Automated Verification Suite & Test Suite Breakdown (33 Tests)

To guarantee 100% mathematical precision and reproducibility, our project includes **33 automated unit tests** covering every equation, statistical distribution, queueing solver, and simulation engine. The entire verification suite runs in **0.19 seconds** via python scripts/verify_all.py.

Test Module	Tests	Key Validations & Tolerances
`test_queueing.py`	6	Erlang-B recursion, Erlang-C, Allen-Cunneen M/G/c, M/M/c/K log-sum-exp, 2-class priority
`test_distributions.py`	4	MLE fitting for Exp/Gamma/Lognormal/Weibull, K-S test p-values, AIC/BIC rankings
`test_arrival_process.py`	4	IDC overdispersion calculation, sinusoidal NHPP fit, Lewis-Shedler thinning accuracy
`test_markov_chain.py`	3	Canonical decomposition P=[Q R; 0 I], fundamental matrix N=(I-Q)^-1, absorption B=NR
`test_simulation.py`	3	Event-calendar DES engine (heapq), priority queue preemption, empirical vs analytical match
`test_jackson_network.py`	4	Solves (I - P^T) Lambda = gamma, station stability rho_k < 1, bottleneck identification
`test_time_varying.py`	3	Diurnal PSA traffic intensity rho(t), wait time profile, dynamic shift staffing gains
`test_bed_allocation.py`	3	Fox's greedy marginal allocation optimizer, convexity validation, budget constraint
`test_mimic_adapter.py`	3	CSV ingestion, ESI triage extraction, department parameter JSON export

4. How Teammates Can Clone, Replicate, and Run Everything

5. 15-Minute Presentation Guide (Sir's Exact Syllabus Structure)

Time	Slide Topic	Key Talking Points & Evidence to Show
Min 1	Problem Definition	Hospital operational tradeoff: bed shortages vs capital waste; emergency boarding.
Min 2	Real-World Data	250,000 real NYS SPARCS inpatient admissions + open MIMIC-IV Clinical Demo.
Min 3-4	Mathematical Theory	Rejection of memoryless assumption (cv^2=2.0); Allen-Cunneen 50% delay penalty; Erlang-B; Jackson network.
Min 5-6	Computational Engine	Modular Python architecture, 33 unit tests (100% passing), DES engine (heapq), Lewis-Shedler thinning.
Min 7-9	Key Findings & Figures	Show Fig 1 (LOS distribution), Fig 2 (Erlang-B buffer), Fig 8 (Dynamic staffing 97% delay cut), Fig 9 (Optimal allocation).
Min 10	Policy Implications	Safety buffer of 24 beds; shifting 25 night beds cuts peak afternoon delay by 97.3%; Fox saves 85.1% blocking.
Min 11	Surge & Limits	+20% winter demand surge triggers collapse (rho=111%); automated elective surgery pause trigger at 95% occupancy.
Min 12	Team Contributions	Teammate 1 (Math & Queueing), Teammate 2 (Literature Review), You (Data Engineering & Simulation).
Min 13-15	Q&A; & Live Demo	Run 'python scripts/verify_all.py' live in terminal to demonstrate 33 tests passing in 0.2s.